

OSLAVA EVENTS

Staff Management Mobile Application

COMPLETE PRODUCT BLUEPRINT

Android-first Flutter application | iOS-ready architecture

Client / Company	Oslava Events
Primary purpose	Automate event staffing, tiered recruitment, attendance and worker quality management
Technology direction	Flutter + Supabase + Firebase Cloud Messaging
Blueprint status	Business rules finalized for architecture and implementation planning

Blueprint intent

This document consolidates the original mobile-app specification and the finalized decisions from the product discussion. It is the reference for UX design, database design, backend rules, and Codex implementation.

Contents

1. Product Vision and Business Context
2. Final Scope and Product Principles
3. Terminology and Domain Model
4. Roles, Authority and Permission Model
5. Worker Registration, Identity and Profile
6. Account States and Detention
7. Event Model and Creation Workflow
8. Event Requirements and Allowances
9. Event Lifecycle
10. Cascading Tier Release Engine
11. Worker Event Discovery and Locked Events
12. Apply-for-Work and Automatic Confirmation
13. Capacity, Concurrency and Priority Allocation
14. Waitlist and Vacancy Refill
15. Cancellation Policy
16. Conflicting Booking Rule
17. Field Operations: Attendance
18. Performance Rating and Category Changes
19. Worker Reliability Score
20. Notifications and Alerts
21. Role-Based App Navigation and Screen Blueprint
22. Technical Architecture
23. Conceptual Data Architecture
24. Security, RBAC and Row Level Security
25. Realtime, Scheduled Jobs and Concurrency
26. Auditability and History
27. Reports and Exports
28. Edge Cases and Failure Handling
29. MVP Scope and Deferred Features
30. Implementation Roadmap for Codex
31. Acceptance Scenarios
- Appendix A. Finalized Business Rules
- Appendix B. Glossary

1. Product Vision and Business Context

Oslava Events currently coordinates a large part-time event workforce using manual messaging and human follow-up. The product goal is to replace that operational overhead with one structured mobile application that manages registration, worker quality tiers, event staffing, automatic tier-based vacancy release, confirmation, attendance, worker performance, and operational history.

The app is designed around a real staffing pattern: multiple events can run on the same day, workers vary substantially in experience and discipline, and higher-quality workers should receive first access to available work. The application must preserve this operating model while removing repeated WhatsApp broadcasts, manual attendance lists, and ad-hoc tracking.

Primary company	Oslava Events, Malappuram
Primary workforce	Part-time catering / event service workers
Initial scale assumption	Roughly 2,000 workers across A, B, C and F categories
Operational load	Often 4-5 events per day, especially on weekends
Platforms	Android first; iOS from the same Flutter codebase later
Core success measure	An admin can publish work and fill, manage and review the event workforce without manual WhatsApp coordination

1.1 Business outcomes

- Reduce manual broadcasting and repeated individual messaging.
- Give high-performing workers predictable priority access to jobs.
- Fill vacancies automatically while preventing overbooking.
- Give admins live visibility into event staffing and vacancies.
- Give Captains and Supervisors a clean field-operations interface.
- Build a permanent record of attendance, lateness, performance, category changes and reliability.
- Create a scalable operational system that can later support additional automation without redesigning the core data model.

2. Final Scope and Product Principles

2.1 Core V1 capabilities

- Worker self-registration and password creation.
- Automatic numeric Worker ID generation.
- Automatic F-category assignment for every new worker.
- Admin event creation with venue, dates/times, wages, requirements and field leaders.
- Flexible A -> B -> C -> F release schedule.
- Push alerts for newly eligible workers.
- Automatic confirmation if a worker is eligible and a seat is available.
- Concurrency-safe capacity enforcement.
- Waitlist after an event becomes full.
- Cancellation allowed until exactly one hour before reporting time.
- One-hour travel buffer between confirmed events.
- Attendance states: Not Marked, Present, Late, Absent.
- Captain/Supervisor direct promotion and demotion with recorded details.
- Worker reliability score and history.
- Account states including Detained.

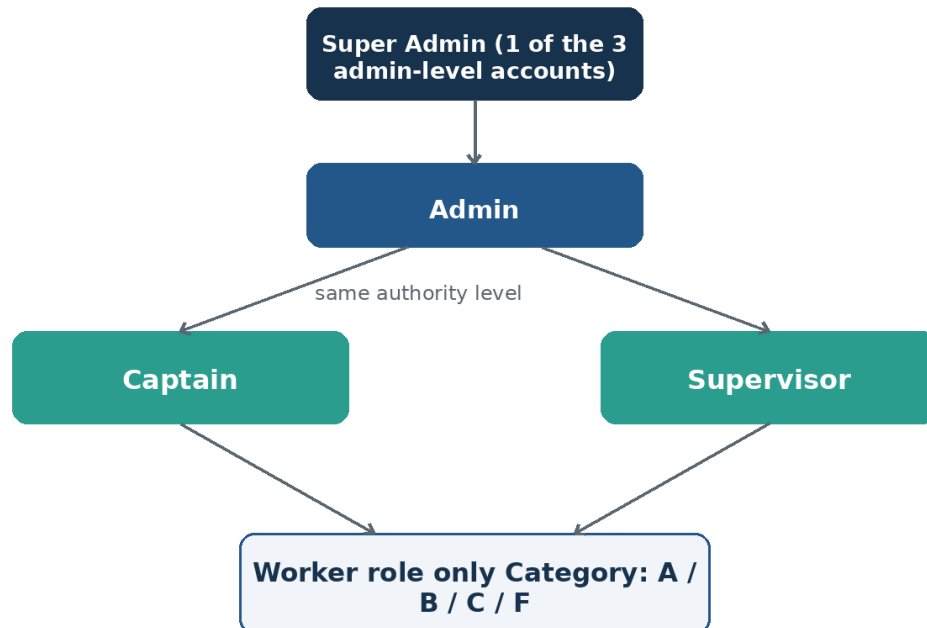
- Role-based UI inside one Flutter application.

2.2 Product principles

Principle	Meaning
Backend is authoritative	Eligibility, booking, priority, capacity, conflict checks, waitlist, account restrictions and category access are decided on the server, not by the Flutter UI.
Roles and categories are separate	Super Admin/Admin/Captain/Supervisor/Worker are roles. A/B/C/F is a category system only for workers.
Realtime where operationally useful	Vacancy counts, confirmations, attendance and event updates should refresh without manual reload where practical.
Audit important actions	Category changes, attendance changes, detention, event edits and role changes must be traceable.
One app, multiple experiences	The same application shows different navigation and permissions according to the logged-in role.
Configurable rather than hardcoded	Tier release timings and event requirements are stored per event or by reusable presets.

3. Terminology and Domain Model

Role and Workforce Category Model



Important: Captain and Supervisor are roles, not worker categories. Only workers belong to A/B/C/F.

Figure 1. Final distinction between user roles and worker categories.

3.1 User roles

- **Super Admin** - one of the admin-level accounts with all Admin capabilities plus permission to manage other Admin accounts.
- **Admin** - creates and manages events, workers and field-leader assignments, but cannot manage the Super Admin.
- **Captain** - field-operations leader assigned to events; equal authority level to Supervisor for worker attendance and performance actions.
- **Supervisor** - field-operations leader assigned to events; a different role label from Captain but the same operational authority level in V1.
- **Worker** - service worker who registers, belongs to one worker category, views events and applies for work.

3.2 Worker categories

Category	Business meaning	Recruitment priority
A	Elite / most trusted and experienced workers	1 - highest
B	Experienced high-performing workers	2
C	Standard / developing workers	3
F	Freshers and all newly registered workers	4 - last

Final rule

Captain and Supervisor do not belong to A/B/C/F. Categories apply only to users whose active role is Worker.

4. Roles, Authority and Permission Model

There are three admin-level accounts initially. One is the Super Admin; the others are normal Admins. The schema should support changing named people later rather than hardcoding individual names.

Capability	Super Admin	Admin	Captain	Supervisor	Worker
Manage Admin accounts	Yes	No	No	No	No
Create / edit / cancel event	Yes	Yes	No	No	No
Assign Captains / Supervisors	Yes	Yes	No	No	No
View all workers	Yes	Yes	Yes	Yes	No
View worker history	Yes	Yes	Yes	Yes	Own only
Mark event attendance	Yes	Yes	Yes	Yes	No
Rate workers	Yes	Yes	Yes	Yes	No
Promote / demote worker category	Yes	Yes	Yes	Yes	No

Detain / release worker	Yes	Yes	Recommended yes	Recommended yes	No
View all events	Yes	Yes	Assigned / relevant	Assigned / relevant	Visible according to event board rules
Apply for work	No	No	No	No	Yes
Join waitlist	No	No	No	No	Yes

Implementation note

Captain and Supervisor may directly promote or demote a worker. No Admin approval step is required. The action must still create a permanent history/audit record.

5. Worker Registration, Identity and Profile

5.1 Registration flow

1. User selects Worker registration.
2. User enters required personal details and creates a password.
3. System validates unique phone number and required fields.
4. Backend creates the authentication account and worker profile.
5. Backend generates a unique numeric Worker ID.
6. Worker category is automatically set to F.
7. Account state is automatically set to ACTIVE.
8. Worker enters the app immediately; no Admin approval is required.

5.2 Registration fields

Field	Required	Notes
Profile photo	Yes	Used by Admin/field leaders for identification.
Full name and initials	Yes	Displayed across event lists and history.
Phone / WhatsApp number	Yes	Primary contact data; should be unique.
Date of birth	Yes	Stored as date, not free text.
Address / native place	Yes	Malappuram-region location information.
Height	Yes	Operationally important for presentation/grooming.
Education status	Yes	Class / course / current education.
Previous experience	Yes	Yes/No plus optional details/reference.
Password	Yes	Created during registration.
Worker category	System	Always F at registration; not editable by worker.

Numeric Worker ID	System	Generated after successful registration.
-------------------	--------	--

5.3 Login

V1 login should use numeric Worker ID (or admin/field-user identifier) plus password. Phone-based OTP is intentionally not required in V1. Password reset can be implemented separately without changing the core identity model.

6. Account States and Detention

State	Meaning	Can apply?
ACTIVE	Normal account	Yes, subject to event rules
SUSPENDED	Temporarily disabled by management	No
DETAINED	Operational penalty / restriction requiring Admin decision	No by default
BLACKLISTED	Severe restriction; management-controlled	No
INACTIVE	User no longer actively working	No

The planned operational direction is that a no-show can lead to detention affecting future work, but V1 will not automatically enforce a fixed number of events. The system will simply show DETAINED and allow management to choose the action or duration.

6.1 Detention record

- Reason
- Related event, if applicable
- Marked by user ID and role
- Date/time
- Optional notes
- Release date or manual-release flag if management chooses one later

7. Event Model and Creation Workflow

Super Admin and Admin can create events. Each event can have multiple Captains and multiple Supervisors. Wages and allowances are informational in V1; the app does not process payroll or payments.

7.1 Final event creation fields

Field	Purpose
Event title	Example: Wedding Reception
Event type	Wedding catering, reception, corporate, etc.
Venue name	Human-readable venue
Google Maps location	Clickable location / coordinates or Maps link
Event date	Calendar date
Reporting time	Mandatory; drives cancellation deadline

Work start time	Expected actual work start
Expected end time	Required for booking-conflict check
Required worker count	Total staffing target
Daily wage	Displayed to workers
Additional allowances	Structured or requirement-linked
Captains	Zero or more assigned users
Supervisors	Zero or more assigned users
Custom requirements	Structured requirements list
Instructions	Free-text operational notes
Dress code	Structured/free-text presentation rules
Cancellation policy	Default: locked 1 hour before reporting
Tier release strategy	Standard / Urgent / Emergency / Custom

8. Event Requirements and Allowances

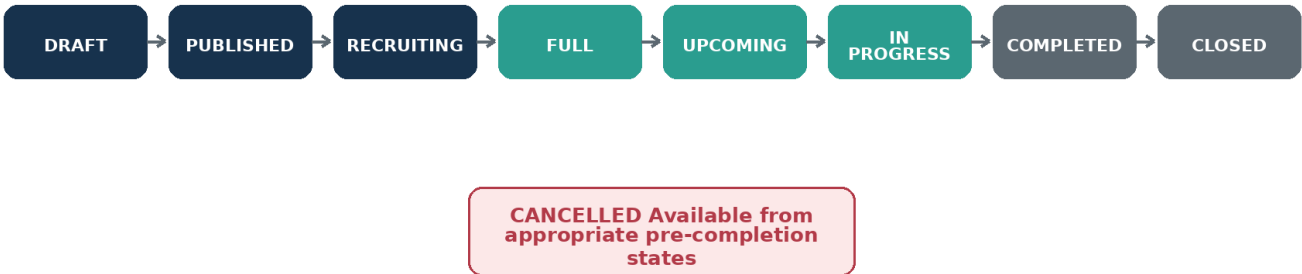
Requirements should not be hidden inside one long description. Each event can have a list of structured requirements so the app can display them clearly and optionally require a confirmation checkbox before application.

Requirement field	Example
Name	Formal shoes
Mandatory	Yes
Worker acknowledgement required	Yes
Extra allowance	INR 50
Description	Black formal shoes in good condition

Worker event detail example: Formal shoes - Required - + INR 50 allowance. Before applying, the worker confirms that mandatory equipment/clothing requirements are satisfied.

9. Event Lifecycle

Event Lifecycle



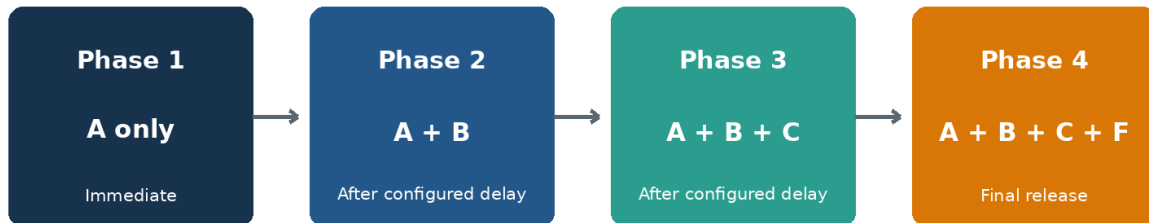
Tier access (A/B/C/F) is tracked separately from the event status.

Figure 2. Event status lifecycle. Tier access is maintained as a separate state.

Status	Meaning
DRAFT	Created but not visible to workers.
PUBLISHED	Published and ready to enter recruitment flow.
RECRUITING	At least one tier is eligible and vacancies remain.
FULL	Confirmed assignments reached required worker count.
UPCOMING	Recruitment complete or near event time; ready for operations.
IN_PROGRESS	Event is currently running.
COMPLETED	Operational work has ended; attendance/reviews may still be finalized.
CLOSED	Administrative close; final records retained.
CANCELLED	Event cancelled by authorized management.

10. Cascading Tier Release Engine

Cascading Tier Release Engine



Higher tiers never lose access when lower tiers open. Timing is selected per event: Standard, Urgent, Emergency, or Custom.

Figure 3. Accumulative category eligibility through the tier-release phases.

Each event stores a tier-release plan. Higher categories retain access permanently once the event is published. Moving to a lower tier expands eligibility; it never removes access from previously eligible tiers.

10.1 Release presets

Preset	A	B	C	F	Use case
Standard	0 min	+30 min	+60 min	+180 min	Normal planned staffing
Urgent	0 min	+15 min	+30 min	+60 min	Short lead time
Emergency	0 min	+5 min	+10 min	+15 min	Immediate staffing need
Custom	Admin-defined	Admin-defined	Admin-defined	Admin-defined	Event-specific strategy

Preset values above are recommended defaults and can be adjusted later without changing architecture. The important finalized rule is flexible timing with accumulative eligibility.

10.2 Timer behavior

- When an event is published, A opens immediately unless custom configuration specifies otherwise.
- At each release threshold, backend checks whether vacancies remain.
- If vacancies remain, the next tier becomes eligible and receives a push notification.
- If the event is already full, the backend should not send a new vacancy alert to the next tier.
- If a vacancy reopens later, current eligible tiers and/or waitlist rules apply.

11. Worker Event Discovery and Locked Events

Workers can see events that are not yet open to their category. This keeps the job board transparent while preserving tier priority.

Event state for worker	Worker experience
Available	Full details plus active Apply button.
Locked for category	Full event details visible; Apply disabled; shows which categories are open and optional countdown to own tier.
Full	Shows full status and Join Waitlist option.
Confirmed	Appears in My Work with cancellation deadline and field-leader information.
Completed	Appears in work history.
Cancelled	Shows cancellation status and related alert.

12. Apply-for-Work and Automatic Confirmation

Application confirmation is automatic. There is no manual Admin approval for normal worker applications.

12.1 Backend validation sequence

9. Verify worker account is ACTIVE and not restricted.
10. Verify the event is recruitable and not cancelled/closed.
11. Verify the worker category is currently eligible.
12. Verify the worker is not already confirmed/applied for the same event.
13. Verify mandatory requirements have been acknowledged.
14. Verify the worker has no conflicting confirmed event within the required one-hour gap.
15. Lock/check event capacity in a concurrency-safe database operation.
16. If capacity remains, create confirmed assignment immediately.
17. If capacity is full, offer or create waitlist entry according to product behavior.
18. Return authoritative result to Flutter: Confirmed, Full/Waitlist, Locked, Conflict, Restricted, or Error.

Critical architecture rule

Flutter never decides whether a seat is available. The server/database transaction is the single authority for assignment.

13. Capacity, Concurrency and Priority Allocation

Capacity must be safe when many workers tap Apply at the same time. The system cannot rely on the vacancy count shown on a phone because that value can become stale.

- Use a database transaction or PostgreSQL function to check and allocate the seat atomically.
- Never allow confirmed assignments to exceed event.required_worker_count.
- First-come-first-served applies within the same worker tier.
- If simultaneous requests from different eligible tiers compete for the final seat, higher category wins: A > B > C > F.
- Once required count is reached, event changes to FULL in realtime.

- Admin and field dashboards receive updated confirmed/vacancy counters.

Exact microsecond tie-breaking implementation will be finalized in the database design, but priority must be implemented server-side, not in the mobile client.

14. Waitlist and Vacancy Refill

When the event becomes full, additional eligible workers can join a waitlist instead of receiving only a rejection.

14.1 Waitlist ordering

- Primary priority: worker category $A > B > C > F$.
- Secondary priority: earliest valid waitlist join time within the same category.
- Account must remain eligible and unrestricted at the time of promotion.
- Conflict check must be run again before converting a waitlist entry into a confirmed assignment.

14.2 Automatic refill

- Confirmed worker cancels before the cancellation deadline or is removed by authorized management.
- Confirmed count falls below required count.
- Backend selects highest-priority valid waitlist entry.
- Backend performs account, category and conflict checks.
- Worker is automatically promoted from waitlist to confirmed.
- Worker receives push notification and in-app alert.
- Dashboard returns to full if target count is restored.

15. Cancellation Policy

Final rule

A worker can cancel a confirmed job only until exactly 1 hour before the event reporting time.

Cancellation deadline = reporting_time - 1 hour. The rule is independent of how long ago the worker applied.

Scenario	Result
Reporting 4:00 PM; worker cancels at 2:45 PM	Allowed
Reporting 4:00 PM; worker cancels at 3:00 PM	Allowed at the boundary if backend time $\leq$ deadline
Reporting 4:00 PM; worker tries at 3:01 PM	Blocked
Worker applies after cancellation deadline	Show late-booking warning; if accepted, cancellation is already locked

After a valid cancellation, the vacancy is refilled from the waitlist when possible. Cancellation events are recorded in worker history and may contribute to reliability metrics.

16. Conflicting Booking Rule

Final rule

There must be at least 1 hour between the expected end of one confirmed event and the reporting time of another confirmed event.

Existing event	New event	Decision
Ends 4:00 PM	Reports 5:00 PM	Allowed
Ends 4:30 PM	Reports 5:00 PM	Blocked - only 30 minutes gap
Ends 6:00 PM	Reports 5:00 PM	Blocked - overlap
Ends 1:00 PM	Reports 5:00 PM	Allowed

The conflict check runs inside the server-side application transaction. A worker may accept multiple events on the same date only when all confirmed assignments satisfy the one-hour buffer.

17. Field Operations: Attendance

Captains and Supervisors share the field-operations capability. They see assigned events and confirmed worker lists and can mark attendance in realtime.

Attendance state	Meaning
NOT_MARKED	No attendance decision yet.
PRESENT	Worker reported and is present.
LATE	Worker reported after expected reporting time.
ABSENT	Worker did not report / confirmed no attendance.

- Field list supports search and category grouping.
- Dashboard shows total confirmed, Present, Late, Absent and Not Marked counts.
- Attendance updates feed worker history and reliability metrics.
- All changes record who marked the status and when.
- Admin can view the same operational data and may be permitted to correct mistakes.

18. Performance Rating and Category Changes

Captain and Supervisor can directly promote or demote a worker. No Admin approval is required. Management users may also change categories if permitted.

18.1 Promotion path

Normal upward path: F -> C -> B -> A. The interface should default to one-step promotion but architecture can support explicit authorized transitions if the business later requires exceptions.

18.2 Demotion path

Normal downward path: A -> B -> C -> F. Demotion requires a reason. Example reasons: Lack of Interest, Misbehavior, Late Arrival, Uniform Issue, Poor Performance, Other.

18.3 Required history record

Field	Example
Worker	Rahul K.P.
Old category	C
New category	B

Action	Promotion
Changed by	Captain Anwar
Related event	Wedding Reception
Reason	Excellent performance
Notes	Handled VIP section very well
Timestamp	System-generated

The worker profile updates immediately after the category change, so future event eligibility follows the new category.

19. Worker Reliability Score

Reliability is a separate operational metric from category. Category controls recruitment priority; reliability summarizes actual behavior and helps field/admin staff make better decisions.

19.1 Inputs

- Attendance rate
- Number of completed events
- Late arrivals
- Absences / no-shows
- Valid cancellations
- Performance ratings
- Recent trend / recency if introduced later

Recommended V1 display: score from 0-100 plus the underlying raw metrics. The exact weighting should be configurable and finalized during database/business-rules implementation rather than hardcoded into the UI.

Important

Reliability does not automatically promote or demote a worker in V1. Captain/Supervisor remains the decision-maker for category changes.

20. Notifications and Alerts

Notification	Audience	Purpose
NEW_EVENT	Worker	A new event is available to the worker category.
TIER_OPENED	Worker	A previously locked event is now open to the worker category.
APPLICATION_CONFIRMED	Worker	Application was accepted automatically.
WAITLIST_JOINED	Worker	Worker joined event waitlist.
WAITLIST_PROMOTED	Worker	Worker moved from waitlist to confirmed.
CAPTAIN_ASSIGNED	Captain	User was assigned as Captain for an

		event.
SUPERVISOR_ASSIGNED	Supervisor	User was assigned as Supervisor for an event.
EVENT_FULL	Admin / field leaders	Required worker count has been reached.
EVENT_UPDATED	Affected users	Time, venue, requirement or other significant event detail changed.
EVENT_CANCELLED	Affected users	Event cancelled.
REPORTING_REMINDER	Confirmed worker	Reminder before reporting time.
CATEGORY_CHANGED	Worker	Worker was promoted or demoted.
ACCOUNT_DETAINED	Worker	Account status changed to Detained.
VACANCY_REOPENED	Eligible users / waitlist	A vacancy became available again.

Push delivery will use Firebase Cloud Messaging, while notification state/history can be stored in Supabase. Scheduled tier releases and reminders are backend jobs; they must not depend on the mobile app being open.

21. Role-Based App Navigation and Screen Blueprint

One Flutter application presents different navigation after authentication according to the user role.

21.1 Worker navigation

Tab	Key screens / actions
Home	Category, reliability, next confirmed work, available jobs, urgent notices
Events	Available, locked, full/waitlist, upcoming job cards
My Work	Confirmed, completed, cancelled work; cancellation countdown/deadline
Alerts	Notification history
Profile	Personal details, category, reliability, attendance/work history, category history

21.2 Captain / Supervisor navigation

Tab	Key screens / actions
Home	Today's assigned events, staffing status, field alerts
Events	Assigned event details and worker lists
Workers	Search all workers, profile/history, rating, promotion/demotion
Alerts	Assignment and event notifications
Profile	Own account details

21.3 Admin navigation

Tab	Key screens / actions
Dashboard	Today's events, slots required/filled, worker category counts, operational alerts
Events	Calendar/list, create, edit, cancel, assign field leaders, export lists
Workers	Search/filter, worker details, category/history/account status
Team	Manage Captains and Supervisors; Super Admin also manages Admin accounts
More	Settings, notification presets, audit/history, reports

21.4 Critical worker event detail screen

- Event title and type
- Venue + open in Google Maps
- Date, reporting time, work start/end
- Wage and allowances
- Required dress/equipment
- Captain/Supervisor names and contact when confirmed
- Vacancy count
- Tier lock / countdown or Apply button
- Join Waitlist when full
- Cancellation deadline for confirmed work

22. Technical Architecture

Recommended Technical Architecture

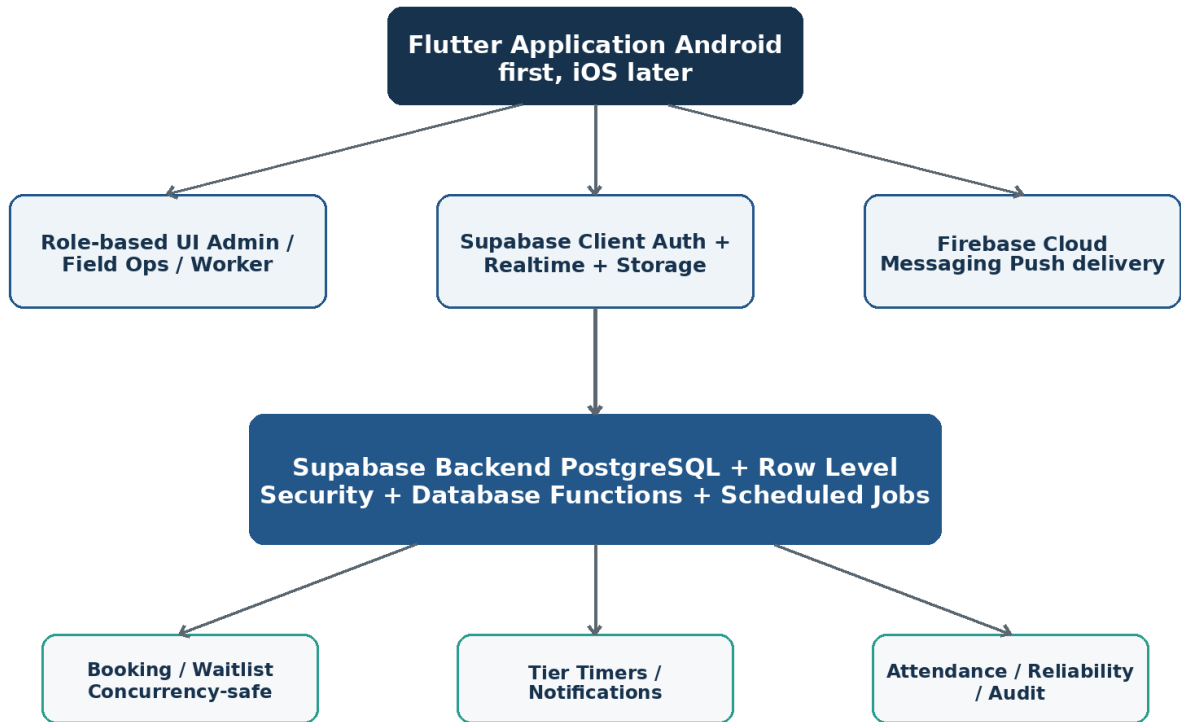


Figure 4. Recommended high-level implementation architecture.

22.1 Technology stack

Layer	Technology	Responsibility
Mobile UI	Flutter / Dart	Android-first application and future iOS build
State / navigation	Riverpod + go_router	Predictable app state and role-based routing
Backend platform	Supabase	Auth, PostgreSQL, Realtime, Storage, server functions
Database	PostgreSQL	Authoritative operational data and transactional rules
Push	Firebase Cloud Messaging	Android/iOS push delivery
Maps	Google Maps deep link / integration	Venue navigation
Source control	Git + GitHub	Version history and safe Codex workflow
Coding workflow	Codex	Primary implementation agent, tests and refactoring

22.2 Client vs backend responsibilities

Flutter client	Backend / database
Render screens	Authorize data access
Collect forms	Validate event eligibility
Show countdowns/status	Enforce cancellation deadline
Display vacancy counts	Enforce exact capacity
Request Apply	Allocate seat transactionally
Request category change	Record/validate authorized category change
Receive realtime updates	Resolve waitlist and conflicting bookings
Display notifications	Schedule tier releases and reminders

23. Conceptual Data Architecture

This blueprint defines the conceptual entities. Exact PostgreSQL column types, indexes, constraints and RLS policies will be produced in the dedicated Supabase schema design step.

Entity	Purpose
profiles	Core identity, numeric ID, role, phone, display info, account status
worker_profiles	Worker-only attributes including category and reliability summary
worker_category_history	Permanent promotion/demotion history
events	Event details, capacity, status, dates/times, wages and release strategy
event_leaders	Many-to-many assignment of Captains/Supervisors to events
event_requirements	Structured dress/equipment requirements and allowances
event_tier_release_rules	Per-event timing for A/B/C/F access
applications / assignments	Worker application and confirmed staffing records
waitlist	Priority-ordered overflow candidates
attendance	Per-assignment attendance state and audit metadata
performance_reviews	Ratings and event-based worker feedback
cancellations	Cancellation history and timing
detentions / account_actions	Operational restrictions and management decisions
notifications	In-app notification records and delivery state
audit_logs	Security/operations history for critical changes

23.1 Core relationships

- One Worker -> many applications/assignments.

- One Event -> many assignments and waitlist entries.
- One Event -> many requirements.
- One Event -> many field leaders through event_leaders.
- One Worker -> many attendance records through assignments.
- One Worker -> many performance reviews and category-history records.
- One Event -> one configurable tier-release plan or linked preset.

24. Security, RBAC and Row Level Security

The mobile app is distributed to workers, so security cannot rely on hiding buttons. Supabase Row Level Security and server-side functions must enforce every sensitive rule.

- Worker can read/edit own profile fields allowed by policy, but cannot change role, category, reliability or account status.
- Worker can read worker-visible event data and own applications/assignments.
- Worker cannot directly insert a confirmed assignment; must call controlled Apply function.
- Captain/Supervisor can access field data required for their role and worker management scope.
- Admin can manage company events/workers according to policy.
- Only Super Admin can create/revoke Admin-level accounts.
- Service-role credentials never ship inside Flutter.
- Critical mutations validate the caller role server-side.

24.1 Sensitive fields

Role, worker category, account status, detention state, reliability aggregate, attendance audit fields, event capacity and assignment state must never be freely client-editable.

25. Realtime, Scheduled Jobs and Concurrency

25.1 Realtime use cases

- Confirmed / required worker count on Admin dashboard
- Worker status changing from waitlist to confirmed
- Event becoming full or reopening
- Attendance counters during event day
- Significant event edits
- Category/account-status changes reflected in the worker session

25.2 Scheduled backend jobs

- Tier B/C/F release according to event timing rules
- Reporting reminders
- Optional event status transitions
- Cleanup/retry of notification delivery
- Future detention/suspension expiration if automated later

25.3 Concurrency controls

- Atomic seat allocation
- Unique assignment per worker/event
- Capacity cannot exceed target
- Waitlist promotion lock
- Category-priority tie resolution
- Transaction-safe conflict recheck

26. Auditability and History

Because Captains and Supervisors can directly affect a worker career tier, the app must preserve strong history even though no Admin approval is required.

Action	Audit requirement
Promotion / demotion	Old/new category, reason, actor, role, event, timestamp
Attendance edit	Old/new attendance, actor, timestamp
Detain/release	Reason, actor, duration/manual state, timestamp
Event edit/cancel	Changed fields, actor, timestamp
Admin role change	Super Admin actor, old/new role, timestamp
Captain/Supervisor assignment	Event, user, assigned/removed by, timestamp
Manual assignment removal	Reason, actor, timestamp

27. Reports and Exports

The original operating requirement includes generating neat event staffing lists that can be shared with site/kitchen teams. This should remain in the product roadmap and can be implemented without changing the core model.

- Confirmed worker list grouped/sorted by category.
- Attendance list with Present/Late/Absent.
- PDF export for event staffing or attendance.
- Plain-text/shareable summary for messaging apps.
- Worker history summary for management use.
- Category distribution and event staffing analytics.

Payroll settlement and payment processing are out of V1 scope. Wage and allowances are displayed and stored only as event information.

28. Edge Cases and Failure Handling

Edge case	Expected behavior
Worker applies as final seat is taken	Backend returns Full/Waitlist; no overbooking.
A and C compete for final seat nearly simultaneously	Server gives priority to A when tie-resolution criteria apply.
Worker is promoted while event is recruiting	New category affects future eligibility checks; current lock state refreshes.
Worker becomes detained after confirmation	Management must decide whether existing confirmed assignment remains; surface warning to Admin.
Admin reduces required count below confirmations	Block reduction or require explicit removal workflow; never silently delete assignments.
Event time changes and creates booking conflict	Warn affected worker/Admin and flag conflict for resolution.
Event becomes full before next tier timer	Do not send next-tier vacancy notification.
Confirmed worker cancels before deadline	Cancel, audit, refill from waitlist.

Worker tries to cancel after deadline	Block and explain the reporting obligation.
Phone temporarily offline	Show cached UI carefully but all final booking decisions require backend response.
Duplicate Apply taps	Idempotent backend prevents duplicate assignment.
Captain/Supervisor changes category twice	Each change is separately audited; UI should display latest category immediately.

29. MVP Scope and Deferred Features

29.1 MVP must include

- Authentication and role-based routing
- Worker registration/profile/numeric ID
- A/B/C/F categories
- Admin event creation/edit/cancel
- Multiple Captains/Supervisors per event
- Structured requirements and wage display
- Tier-release presets/custom timing
- Worker job board including locked events
- Automatic application confirmation
- Capacity-safe booking and conflict checks
- Waitlist and refill
- One-hour-before-reporting cancellation
- Attendance including Late
- Performance reviews and direct category changes
- Reliability score display
- Account states including Detained
- Core push notifications
- Basic exports/history

29.2 Deferred / future enhancements

- QR attendance
- GPS/geofence check-in
- Automated payroll/payment settlement
- Advanced financial module
- Face recognition
- In-app chat
- AI staffing recommendations
- Automatic category promotion based on score
- Automated next-two-event detention policy
- Multi-company SaaS tenancy if Oslava later turns the product into a platform

30. Implementation Roadmap for Codex

Phase	Area	Outcome
0	Project foundation	Flutter structure, theme, Riverpod, go_router, environments, Git

1	Supabase foundation	Auth setup, schema migrations, base RLS, seed roles
2	Identity	Registration, login, numeric IDs, profile, account states
3	Worker domain	Category model, history, worker search/profile
4	Event management	Create/edit/cancel, leaders, requirements, calendar/list
5	Worker job board	Available/locked/full views and detail screens
6	Booking engine	Server-side Apply function, capacity, category eligibility
7	Conflict + cancellation	One-hour travel buffer and cancellation deadline
8	Waitlist	Join, ordering, automatic promotion
9	Tier engine	Presets/custom schedule and backend jobs
10	Field Ops	Attendance and realtime counters
11	Performance	Ratings, direct promotion/demotion and history
12	Reliability	Metrics/score calculation and profile display
13	Notifications	FCM, assignment alerts, tier alerts, reminders
14	Reports	PDF/text exports and management summaries
15	QA / hardening	Concurrency tests, RLS tests, offline/error UX, analytics
16	Android release	Signing, privacy, Play Store testing
17	iOS	Mac/Xcode config, APNs, App Store release

Codex should receive one bounded phase/task at a time, with tests and flutter analyze run after each meaningful change. Git commits should mark working milestones.

31. Acceptance Scenarios

Scenario	Acceptance result
Worker registration	New worker registers -> numeric ID generated -> ACTIVE -> category F -> can log in immediately.
Tier cascade	Event published -> A alerted/open -> configured delay -> B opens while A stays open -> later C -> F.
Automatic booking	Eligible worker taps Apply -> server validates -> seat available -> confirmed instantly -> counter updates.

Full event	Target reached -> event FULL -> Apply disabled -> Join Waitlist available.
Waitlist refill	Confirmed worker cancels in time -> highest-priority valid waitlist worker confirmed automatically.
Conflict prevention	Worker has event ending 4:30 PM and new reporting at 5:00 PM -> application blocked due to <1 hour gap.
Valid second event	Existing event ends 4:00 PM; next reports 5:00 PM -> application may proceed.
Cancellation	Reporting 4:00 PM -> cancel at/before 3:00 PM accepted; after 3:00 PM blocked.
Attendance	Captain marks worker Late -> event counts and worker history update.
Promotion	Supervisor changes C -> B with details -> no Admin approval -> history record created -> new category used for future events.
Detention	No-show worker marked Detained -> status visible and further application blocked until management changes state.
Admin control	Normal Admin cannot manage Admin accounts; Super Admin can.

Appendix A. Finalized Business Rules

26. One Flutter application for Android first and iOS later.
27. Three admin-level accounts initially; one is Super Admin and can manage Admin accounts.
28. Captain and Supervisor are different roles at the same operational authority level.
29. Only workers belong to A/B/C/F categories.
30. Every newly registered worker starts in F and can work immediately without approval.
31. Worker creates a password and receives a numeric ID.
32. Captain and Supervisor can directly promote/demote workers without Admin approval; details are mandatory/audited.
33. Event can have multiple Captains and multiple Supervisors.
34. Event form includes venue, maps location, dates/times, count, wage, allowances, requirements, leaders and tier strategy.
35. Requirements are structured and can include allowances and worker acknowledgement.
36. Tier release timing is flexible with Standard/Urgent/Emergency/Custom options.
37. Higher tiers retain access after lower tiers open.
38. Workers can see locked events before their category is eligible.
39. Application confirmation is automatic when backend rules pass.
40. Backend enforces higher-tier priority and exact capacity.
41. Full events support a priority waitlist.
42. Cancellation is allowed only until 1 hour before reporting time.
43. At least 1 hour is required between the end of one confirmed event and reporting time of another.
44. Attendance includes Late in addition to Present/Absent/Not Marked.
45. Worker reliability score is included but does not automatically change category in V1.
46. Account state includes Detained; V1 leaves detention handling/duration to management.
47. Wages/allowances are displayed only; payroll/payment processing is not part of V1.

Appendix B. Glossary

Term	Definition
Admin-level account	Super Admin or Admin.
Field Ops	Captain/Supervisor operational interface for event-day workforce management.
Category	Worker-only quality/priority tier: A, B, C or F.
Locked event	Visible event that is not yet open to the worker category.
Confirmed assignment	Authoritative backend record that reserves a worker seat for an event.
Waitlist	Ordered overflow list used to refill a vacancy after event capacity is reached.
Reporting time	Time the worker must arrive; cancellation locks one hour before this time.
Work end time	Expected end of work; used in one-hour event-conflict buffer.
Detained	Management-controlled account restriction after an operational issue such as no-show.
Reliability score	Operational summary metric separate from worker category.

Blueprint Status

Ready for technical design

The product/business blueprint is finalized enough to begin the next design layer: exact Supabase schema, database constraints, RLS policies, backend functions, notification jobs and then staged Codex implementation.

The exact SQL schema and security policies are intentionally not embedded as final code in this document; they should be generated as a dedicated architecture artifact so database decisions can be reviewed independently before implementation.